

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-a1a6cac74f170afa1.jsonl`  
- SHA-256: `f34ee85384961e6e30e8ac95dd4ecf8a76271efe9c8ae567285caff2e927c94a`  
- Source modified: `2026-07-06T06:04:56+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:02:53.379Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

DIMENSION: dedup correctness (compute_dedup_key + the Tier-1/Tier-2 shims + content_items uniqueness).

Investigate specifically:

- SEED CLAIM to independently verify or refute: the v1 Tier-2 fingerprint dedup (get_video_by_content_fingerprint) is REGRESSED. v1 did SELECT ... WHERE size/dur/h/w match (matched ANY prior video incl. those with a document_id). The new shim looks up dedup_key='tg:fp:S:D:H:W', but a prior item that HAD a document_id was stored as 'tg:doc:X' ? so the fingerprint lookup misses it. Reproduce it and confirm/deny. Assess real-world severity given nearly all Telegram videos have a document_id and README advertises 3-tier dedup.
- Parity between the Python compute_dedup_key decision tree and the pure-SQL CASE in _migration_003 (field order, guards, provider prefix casing).

- False positives/negatives: partial metadata, empty-string document_id, provider casing, cross-source Tier-0.
- The IntegrityError race handling in telethon_client.py vs the new dual UNIQUE constraints.

Return findings per schema.

2. assistant (2026-07-06T06:02:55.633Z)

I'll start by reading the key files to understand the dedup implementation.

3. user (2026-07-06T06:02:56.394Z)

```
1      # ADR 0011: Provider-neutral domain schema (Track B)
2
3      - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4      - **Date**: 2026-07-06
5      - **Decision owner**: project owner
6      - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7      - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8      - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10     ## Context
11
12     ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13     (MTPProto credential). That unblocked the *identity* dimension. This ADR
14     tackles the **content dimension**: how to represent sources, content items,
15     policies, destinations, routes, and delivery attempts in a provider-neutral
16     way, so that the system can ingest from Telegram today and from WhatsApp
17     (or any future provider) later without another schema rewrite.
18
19     ### What we have today (post-Track-A)
20
21     | Table | Meaning | Provider-coupling |
22     |---|---|---|
23     | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24     | `telegram_sessions` | Optional MTPProto session (ADR 0010) | Telegram-specific by
design |
25     | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26     | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27     | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29     The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30     - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31     - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32     - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
33
34     ### What we need
35
36     The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37     `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38     defines the schema for each, the **two open design questions** that gate the
39     schema, and the migration path from the current tables.
40
41     ## The two decisions I need from you
42
43     ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45     The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46     concept). A provider-neutral `ContentItem` needs a dedup story that works
47     across providers. Three options:
48
```

```

49 ##### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
50
51 Keep ADR 0006's three tiers but generalize them behind a single
52 `dedup_key` column on `content_items`:
53
54 ```
55 content_items.dedup_key TEXT NOT NULL
56 ```
57
58 The *value* of `dedup_key` is computed by an explicit decision tree at
59 ingestion time, provider by provider. The tree is ordered strongest-key
60 first and each branch is guarded so partial metadata can never collapse
61 to a degenerate key* (e.g. `tg:fp:0:0:0:0`):
62
63 - Telegram:
64   1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
65   2. Else if all four fingerprint fields (`size_bytes`, `duration_s`,
66     `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
67   3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
68     (Tier 0; the source/channel id is required because Telegram
69     `message_id` is only unique within a channel, not across sources).
70 - Future WhatsApp: `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
71   ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
72   exposes a content hash.
73
74 The guard in step 2 (all four fields required) is what prevents the
75 partial-metadata collision the first draft of this ADR allowed. The pure
76 `compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
77 size_bytes, duration_s, height, width)` function that implements this tree
78 gets its own unit tests in the implementation PR, including cases for
79 partial/null metadata.
80
81 A partial unique index on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
82 replaces the current Tier-1 index and enforces per-account dedup at the DB
83 level, regardless of provider.
84
85 Why recommended:
86 - Zero new bandwidth ? all keys are computed from metadata already extracted
87   (honors ADR 0003's no-download principle).
88 - The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
89   structurally impossible.
90 - Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
91   existing `document_id` / metadata columns during the migration.
92 - Supersedes ADR 0006 cleanly: the three tiers become one column with
93   provider-specific derivation rules, documented in the migration.
94
95 Costs:
96 - The migration must compute `dedup_key` for every existing `videos` row
97   (cheap ? pure SQL string building, no I/O).
98 - The `dedup_key` derivation logic lives in code (a pure function per
99   provider), not in the schema. This is a feature, but it means the
100   derivation rules need their own unit tests.
101
102 ##### Option 1B: content hash (SHA-256 of downloaded bytes)
103
104 `dedup_key = sha256(downloaded_content)`. Cryptographically strong,
105 provider-agnostic, catches re-encodes that 1A misses.
106
107 Rejected ? violates ADR 0003 (metadata-first, no download in the hot
108 path). Downloading every incoming video to hash it would multiply bandwidth
109 and latency by orders of magnitude. Could be added later as an optional
110 deep-inspect stage (deferred to Track F per #31).
111
112 ##### Option 1C: provider-assigned only (drop Tier 2)
113

```

```

114     `dedup_key` = "tg:doc:{document_id}"` always; no metadata fingerprint
115     fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
116     `document.id` ? a real and common case (someone re-encodes and re-uploads
117     the same video to a different channel).
118
119     **Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly,
and honors the no-download constraint.
120
121     ---
122
123     ### Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?
124
125     The codebase already has `filter_config.config_json` (one row per account,
126     holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The
127     umbrella introduces a `Policy` concept. They overlap. Two options:
128
129     #### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
130
131     `policies` is a new table; `filter_config` is migrated into it. One account
132     can own multiple policies (enables per-route filter overrides from ADR
133     0009), with a `DEFAULT` policy per account for the no-rules-configured case.
134
135     ```
136     policies (
137         id            INTEGER PRIMARY KEY AUTOINCREMENT,
138         account_id    INTEGER NOT NULL REFERENCES accounts(id),
139         name          TEXT NOT NULL,                -- e.g. "default", "reggae-route"
140         config_json   TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
141         is_default    INTEGER NOT NULL DEFAULT 0,    -- one per account (UNIQUE partial
idx)
142         created_at   TEXT NOT NULL,
143         updated_at   TEXT NOT NULL,
144         UNIQUE (account_id, name)
145     );
146     ```
147
148     **Phase 1** (this track): `config_json` holds exactly today's `FilterConfig`
149     JSON ? the rename is purely structural. **Phase 2** (Track F, deferred):
150     `config_json` grows to include topic preferences and security rules, but
151     that's behind the same column so no further migration.
152
153     Migration: for each `filter_config` row, insert a `policies` row with
154     `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop
155     `filter_config` after the migration (keep `filter_config_deprecated` for one
156     release, mirroring ADR 0010's safety-net pattern).
157
158     **Why recommended:**
159     - `Policy` becomes the single home for "what to filter/enrich" ? no second
160       overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
161     - Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`)
162       without a second migration later.
163     - The existing `db.load_filter_config(account_id)` / `save_filter_config`
164       become thin shims over `policies WHERE account_id=? AND is_default=1`,
165       mirroring the Track-A `User` shim pattern ? backwards-compatible.
166
167     **Costs:**
168     - One more table migration. `filter_config` ? `policies` is a rename+extend.
169     - The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape);
170       `Policy` is the *container* (name, owner, default flag) around it. Two
171       models, not one ? but they compose cleanly.
172
173     #### Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer
174
175     Don't touch `filter_config`. `policies` references a `filter_config` row
176     plus adds enrichment/security stages as separate columns.

```

```

177
178  **Rejected** ? creates two overlapping concepts (`filter_config` for the
179  v1 rules, `policies` for "everything else"), which is exactly the
180  conflation the umbrella exists to dissolve. We'd be back here in a year
181  merging them.
182
183  **Recommendation: 2A.** One concept, one table, phased growth via the same JSON column.
184
185  ---
186
187  ## Proposed schema (assuming 1A + 2A)
188
189  ### `sources` (generalizes `channels`)
190
191  A configured place we ingest from. Maps 1:1 to today's per-account
192  `channels` rows for Telegram; future providers add their own source kinds.
193
194  ```
195  sources (
196      id                INTEGER PRIMARY KEY AUTOINCREMENT,
197      account_id        INTEGER NOT NULL REFERENCES accounts(id),
198      provider          TEXT NOT NULL DEFAULT 'telegram',
199      provider_source_id TEXT NOT NULL,          -- Telegram channel_id; future: WA group id
200      title             TEXT,
201      kind              TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
202      watched           INTEGER NOT NULL DEFAULT 0,      -- ADR 0009 selective scanning
203      seen_first_at     TEXT NOT NULL,
204      last_seen_at      TEXT,
205      UNIQUE (account_id, provider, provider_source_id)
206  );
207  ```
208
209  `channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`;
210  `channels.channel_id` maps to `sources.provider_source_id`. The `watched`
211  flag (ADR 0009, deferred to Track E) is included here so Track E doesn't
212  need another migration.
213
214  ### `content_items` (generalizes `videos`)
215
216  A normalized item discovered from any source. Phase 1 stores only videos
217  (matches today's `videos`); the columns are provider-neutral but the
218  Phase-1 migration only populates video fields.
219
220  ```
221  content_items (
222      id                INTEGER PRIMARY KEY AUTOINCREMENT,
223      account_id        INTEGER NOT NULL REFERENCES accounts(id),
224      source_id         INTEGER NOT NULL REFERENCES sources(id),
225      provider          TEXT NOT NULL DEFAULT 'telegram',
226      provider_message_id TEXT NOT NULL,          -- Telegram message_id; future: WA message
227      id
228      content_type      TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' |
229      ...
230      text              TEXT,                      -- caption / post text (nullable)
231      link_url          TEXT,                      -- for link/article posts
232      media_ref         TEXT,                      -- provider media reference (file path,
233      blob id)
234      document_id       TEXT,                      -- Telegram document.id (kept for TG-
235      specific lookups)
236      duration_s        INTEGER,
237      width             INTEGER,
238      height            INTEGER,
239      size_bytes        INTEGER,
240      mime_type         TEXT,
241      dedup_key         TEXT NOT NULL,             -- ADR 0011 Question 1 (Option 1A)

```

```

238         matched          INTEGER NOT NULL DEFAULT 0,
239         seen_at           TEXT NOT NULL,
240         -- Tier-0 equivalent: provider_message_id is only unique within a source
241         -- (a Telegram message_id is per-channel), so source_id is required to
242         -- avoid false collisions across sources for the same account.
243         UNIQUE (account_id, source_id, provider_message_id),
244         UNIQUE (account_id, dedup_key)                -- Tier-1/2 equivalent (Option
1A)
245     );
246     ```
247
248     **Note on `forwarded`:** dropped from `content_items`. Per-destination
249     delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
250     (before Track C lands), `content_items.matched` alone is sufficient ?
251     delivery still goes to the single `delivery_target`, and the boolean
252     `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
253     so the v1 delivery path keeps working. Track C removes it.
254
255     ### `policies` (generalizes `filter_config`) ? Option 2A
256
257     As specified in Question 2 above.
258
259     ### `destinations` (new ? Track C will populate)
260
261     Defined here so the schema is complete, but **Track C (#28) implements the
262     delivery layer**. Phase-1 presence is the table only (no delivery logic).
263
264     ```
265     destinations (
266         id          INTEGER PRIMARY KEY AUTOINCREMENT,
267         account_id  INTEGER NOT NULL REFERENCES accounts(id),
268         provider    TEXT NOT NULL DEFAULT 'telegram',
269         target      TEXT NOT NULL,                -- 'me', '@username', chat id (ADR 0005)
270         kind        TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
271         title       TEXT,
272         is_output   INTEGER NOT NULL DEFAULT 0,    -- ADR 0009: distinguishes output
channels
273         created_at  TEXT NOT NULL,
274         UNIQUE (account_id, provider, target)
275     );
276     ```
277
278     ### `routing_rules` (new ? Track C will populate)
279
280     Per ADR 0009 (canonical name locked in issue #25).
281
282     ```
283     routing_rules (
284         id          INTEGER PRIMARY KEY AUTOINCREMENT,
285         account_id  INTEGER NOT NULL REFERENCES accounts(id),
286         source_id   INTEGER NOT NULL REFERENCES sources(id),
287         policy_id   INTEGER,                      -- nullable ? use account's default policy
288         destination_id INTEGER NOT NULL REFERENCES destinations(id),
289         enabled     INTEGER NOT NULL DEFAULT 1,
290         priority    INTEGER NOT NULL DEFAULT 0,
291         created_at  TEXT NOT NULL,
292         UNIQUE (account_id, source_id, destination_id)
293     );
294     ```
295
296     ### `delivery_attempts` (new ? Track C will populate)
297
298     Per #28's idempotency requirement (unique index on
299     `(content_item_id, destination_id)`).
300

```

```

301     ``
302     delivery_attempts (
303         id                INTEGER PRIMARY KEY AUTOINCREMENT,
304         content_item_id    INTEGER NOT NULL REFERENCES content_items(id),
305         destination_id     INTEGER NOT NULL REFERENCES destinations(id),
306         route_id          INTEGER REFERENCES routing_rules(id),
307         status             TEXT NOT NULL DEFAULT 'pending', --
'pending' || 'delivered' || 'failed' || 'retrying'
308         attempt_count      INTEGER NOT NULL DEFAULT 0,
309         last_error         TEXT,
310         created_at         TEXT NOT NULL,
311         delivered_at       TEXT,
312         next_retry_at      TEXT,
313         UNIQUE (content_item_id, destination_id)           -- ADR #28 idempotency
314     );
315     ``
316
317     ## Migration strategy (integer-preserving, phased)
318
319     This track lands **`sources`, `content_items`, and `policies`** only.
320     `destinations`, `routing_rules`, `delivery_attempts` are defined in the
321     schema (so the ADR is complete) but their tables are created empty and
322     populated by Track C. This keeps Track B focused and reviewable.
323
324     ### Migration `_migration_003_domain_schema` (idempotent)
325
326     1. **Create** `sources`, `content_items`, `policies`, `destinations`,
327       `routing_rules`, `delivery_attempts` (all `IF NOT EXISTS`).
328     2. **Backfill `sources`** from `channels`:
329       `provider='telegram`, `provider_source_id=channel_id`, `watched=1`
330       (ADR 0009 backfill: already-seen channels stay watched).
331     3. **Backfill `content_items`** from `videos`, computing `dedup_key` via the
332       explicit decision tree from Option 1A (every branch reachable, no
333       `COALESCE(...,0)` collapsing partial metadata to a degenerate key):
334       - If `document_id` IS NOT NULL:
335         `dedup_key = 'tg:doc:' || document_id`.
336       - Else if `size_bytes` IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
337       AND width IS NOT NULL:
338         `dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
339       width`.
340       - Else (Tier-0 fallback; partial or missing media metadata):
341         `dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id`,
342         where `source_id` is the joined `sources.id` for the row's legacy
343         `channel_id` and `provider_message_id` is the legacy `message_id`.
344
345     The implementation PR must include migration tests that seed rows hitting
346     each branch (including all-fingerprint-fields-null) and assert no two
347     rows collapse to the same `dedup_key`.
348     4. **Backfill `policies`** from `filter_config`: one row per account,
349       `name='default', `is_default=1`, `config_json` copied verbatim.
350     5. **Rename** (don't drop) `channels` ? `channels_deprecated`,
351       `videos` ? `videos_deprecated`, `filter_config` ? `filter_config_deprecated`.
352       Kept for one release as a rollback safety net (ADR 0010 pattern).
353
354     The legacy `videos`/`channels`/`filter_config` tables and their
355     `db.load_filter_config` / `upsert_channel` / `insert_video` repos stay
356     working as shims over the new tables (same pattern as the Track-A `User`
357     shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
358     `commands.py` keep working without a flag day.
359
360     ## Consequences
361
362     **Positive**
363     - Provider-neutral schema: `sources`, `content_items`, `policies`,
364       `destinations`, `routing_rules`, `delivery_attempts` all carry a

```

```

363     `provider` column or provider-prefixed keys. Adding WhatsApp later is a
364     new `provider='whatsapp'` value, not a new schema.
365 - `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
366     quality and without downloading anything.
367 - `Policy` (Option 2A) gives a single home for filter + future enrichment
368     config, enabling ADR 0009's per-route overrides with no second migration.
369 - Backwards-compatible via shims, mirroring the Track-A pattern that
370     already passed review.
371
372 **Negative**
373 - Six new tables; the schema grows significantly. Each is small and
374     single-purpose, but the surface area is larger.
375 - The dedup-key derivation logic is in code, not the schema ? needs its
376     own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377 - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378     `text`) that aren't populated until later tracks. This is intentional
379     (avoid future migrations) but means some columns are NULL until then.
380
381 ## Alternatives considered
382
383 - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384     providers in code.** Rejected ? same conflation argument as ADR 0010.
385     Telegram-specific ids in column names (`channel_id`, `message_id`,
386     `document_id`) would leak into every future provider's code path.
387 - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388     already the largest track. Splitting schema (Track B) from delivery
389     logic (Track C) keeps each PR reviewable. The empty `destinations` /
390     `routing_rules` / `delivery_attempts` tables in this track are the
391     contract Track C codes against.
392 - **`content_items` as a view over `videos` (no migration).** Rejected ?
393     views don't give us the `provider` column or the `dedup_key` index, and
394     SQLite view support for writes is limited. A real table is needed.
395
396 ## Open questions for the decision owner
397
398 1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399     rejected), or 1C (provider-assigned only).
400 2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401     or 2B (wrap, rejected).
402 3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403     / `text` as NULL until later tracks populate them? Recommendation: yes
404     (avoids a future `ALTER TABLE`).
405 4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406     Recommendation: yes.
407
408 ## References
409
410 - ADR 0003 ? metadata-first probing (the no-download constraint that rules
411     out Option 1B's content hashing).
412 - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413     `routing_rules`).
414 - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415     Option 1A).
416 - ADR 0009 ? selective scanning + multiplexed routing (depends on
417     `sources.watched` and `routing_rules`, both defined here).
418 - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419     at `accounts.id`).
420 - Issue #27 ? Track B (this ADR's tracking issue).
421 - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422

```

4. user (2026-07-06T06:02:57.237Z)

```

1     """SQLite persistence layer (aiosqlite).
2

```



```

3 Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4 the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5 for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6 ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8 The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9 in-memory database (``aiosqlite.connect(":memory:")``).
10 ""
11
12 from __future__ import annotations
13
14 from collections.abc import Awaitable, Callable, Iterable
15 from datetime import UTC, datetime
16
17 import aiosqlite
18
19 from tg_video_filter.models import (
20     Account,
21     Channel,
22     ContentItem,
23     FilterConfig,
24     Policy,
25     Source,
26     TelegramSession,
27     User,
28     VideoRecord,
29 )
30
31 # --- DDL -----
32
33 _SCHEMA = ""
34 CREATE TABLE IF NOT EXISTS accounts (
35     id            INTEGER PRIMARY KEY,
36     telegram_id   INTEGER UNIQUE,
37     provider      TEXT NOT NULL DEFAULT 'telegram',
38     display_name  TEXT,
39     created_at    TEXT NOT NULL
40 );
41
42 CREATE TABLE IF NOT EXISTS telegram_sessions (
43     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44     phone         TEXT,
45     session_path  TEXT NOT NULL,
46     enabled       INTEGER NOT NULL DEFAULT 1,
47     created_at    TEXT NOT NULL,
48     last_login_at TEXT
49 );
50
51 CREATE TABLE IF NOT EXISTS channels (
52     id            INTEGER PRIMARY KEY AUTOINCREMENT,
53     user_id       INTEGER NOT NULL REFERENCES accounts(id),
54     channel_id    INTEGER NOT NULL,
55     title         TEXT,
56     seen_first_at TEXT NOT NULL,
57     UNIQUE (user_id, channel_id)
58 );
59
60 CREATE TABLE IF NOT EXISTS videos (
61     id            INTEGER PRIMARY KEY AUTOINCREMENT,
62     user_id       INTEGER NOT NULL REFERENCES accounts(id),
63     channel_id    INTEGER NOT NULL,
64     message_id    INTEGER NOT NULL,
65     document_id   INTEGER,
66     duration_s    INTEGER,
67     width         INTEGER,

```

```

68         height      INTEGER,
69         size_bytes   INTEGER,
70         mime_type    TEXT,
71         matched       INTEGER NOT NULL DEFAULT 0,
72         forwarded     INTEGER NOT NULL DEFAULT 0,
73         seen_at       TEXT NOT NULL,
74         UNIQUE (user_id, channel_id, message_id)
75     );
76
77     CREATE TABLE IF NOT EXISTS filter_config (
78         user_id        INTEGER PRIMARY KEY REFERENCES accounts(id),
79         config_json    TEXT NOT NULL
80     );
81
82     -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83     -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84     -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
85     -- are created empty here so the schema is complete ? Track C (#28) implements
86     -- the delivery logic that populates them.
87
88     CREATE TABLE IF NOT EXISTS sources (
89         id              INTEGER PRIMARY KEY AUTOINCREMENT,
90         account_id      INTEGER NOT NULL REFERENCES accounts(id),
91         provider        TEXT NOT NULL DEFAULT 'telegram',
92         provider_source_id TEXT NOT NULL,
93         title           TEXT,
94         kind            TEXT NOT NULL DEFAULT 'channel',
95         watched         INTEGER NOT NULL DEFAULT 0,
96         seen_first_at   TEXT NOT NULL,
97         last_seen_at    TEXT,
98         UNIQUE (account_id, provider, provider_source_id)
99     );
100
101     CREATE TABLE IF NOT EXISTS content_items (
102         id              INTEGER PRIMARY KEY AUTOINCREMENT,
103         account_id      INTEGER NOT NULL REFERENCES accounts(id),
104         source_id       INTEGER NOT NULL REFERENCES sources(id),
105         provider        TEXT NOT NULL DEFAULT 'telegram',
106         provider_message_id TEXT NOT NULL,
107         content_type    TEXT NOT NULL DEFAULT 'video',
108         text            TEXT,
109         link_url        TEXT,
110         media_ref       TEXT,
111         document_id     TEXT,
112         duration_s      INTEGER,
113         width           INTEGER,
114         height          INTEGER,
115         size_bytes      INTEGER,
116         mime_type       TEXT,
117         dedup_key       TEXT NOT NULL,
118         matched         INTEGER NOT NULL DEFAULT 0,
119         seen_at         TEXT NOT NULL,
120         UNIQUE (account_id, source_id, provider_message_id),
121         UNIQUE (account_id, dedup_key)
122     );
123
124     CREATE TABLE IF NOT EXISTS policies (
125         id              INTEGER PRIMARY KEY AUTOINCREMENT,
126         account_id      INTEGER NOT NULL REFERENCES accounts(id),
127         name            TEXT NOT NULL,
128         config_json     TEXT NOT NULL,
129         is_default      INTEGER NOT NULL DEFAULT 0,
130         created_at      TEXT NOT NULL,
131         updated_at      TEXT NOT NULL,
132         UNIQUE (account_id, name)

```

```

133 );
134
135 CREATE TABLE IF NOT EXISTS destinations (
136     id            INTEGER PRIMARY KEY AUTOINCREMENT,
137     account_id    INTEGER NOT NULL REFERENCES accounts(id),
138     provider      TEXT NOT NULL DEFAULT 'telegram',
139     target        TEXT NOT NULL,
140     kind          TEXT NOT NULL DEFAULT 'saved',
141     title         TEXT,
142     is_output     INTEGER NOT NULL DEFAULT 0,
143     created_at    TEXT NOT NULL,
144     UNIQUE (account_id, provider, target)
145 );
146
147 CREATE TABLE IF NOT EXISTS routing_rules (
148     id            INTEGER PRIMARY KEY AUTOINCREMENT,
149     account_id    INTEGER NOT NULL REFERENCES accounts(id),
150     source_id     INTEGER NOT NULL REFERENCES sources(id),
151     policy_id     INTEGER REFERENCES policies(id),
152     destination_id INTEGER NOT NULL REFERENCES destinations(id),
153     enabled       INTEGER NOT NULL DEFAULT 1,
154     priority      INTEGER NOT NULL DEFAULT 0,
155     created_at    TEXT NOT NULL,
156     UNIQUE (account_id, source_id, destination_id)
157 );
158
159 CREATE TABLE IF NOT EXISTS delivery_attempts (
160     id            INTEGER PRIMARY KEY AUTOINCREMENT,
161     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
162     destination_id INTEGER NOT NULL REFERENCES destinations(id),
163     route_id      INTEGER REFERENCES routing_rules(id),
164     status        TEXT NOT NULL DEFAULT 'pending',
165     attempt_count  INTEGER NOT NULL DEFAULT 0,
166     last_error    TEXT,
167     created_at    TEXT NOT NULL,
168     delivered_at  TEXT,
169     next_retry_at TEXT,
170     UNIQUE (content_item_id, destination_id)
171 );
172 ""
173
174 # Note: the ``idx_videos_user_document`` partial unique index is created by
175 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
176 # which is present on fresh tables but only added to legacy v1 DBs by the
177 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
178 # DBs that don't yet have the column.
179
180
181 async def init_db(db_path: str) -> None:
182     """Open the database at ``db_path`` and create/migrate the schema.
183
184     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
185     then runs any pending migrations via the ``user_version`` ladder.
186     """
187     async with aiosqlite.connect(db_path) as conn:
188         await _apply_schema(conn)
189
190
191 async def _apply_schema(conn: aiosqlite.Connection) -> None:
192     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193     # and a busy timeout so concurrent access waits instead of failing immediately.
194     await conn.execute("PRAGMA journal_mode = WAL")
195     await conn.execute("PRAGMA foreign_keys = ON")
196     await conn.execute("PRAGMA busy_timeout = 5000")
197     await conn.executescript(_SCHEMA)

```

```

198 # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199 # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200 # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201 # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202 # any *empty* recreations of those legacy tables from a subsequent
203 # ``init_db`` are dropped here so they don't shadow the renamed copies.
204 if await _table_exists(conn, "channels_deprecated"):
205     await conn.execute("DROP TABLE IF EXISTS channels")
206     await conn.execute("DROP TABLE IF EXISTS videos")
207     await conn.execute("DROP TABLE IF EXISTS filter_config")
208 await _apply_migrations(conn)
209 await conn.commit()
210
211
212 # --- migrations -----
213 #
214 # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.
215 # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216 # the source of truth for *fresh* databases; the migrations below reconcile
217 # *existing* databases that were created by an older schema version.
218 #
219 # To add a new migration:
220 # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221 # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
222 # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
223 #    ``init_db``, and asserts the new structure.
224
225 _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
226
227
228 def _migration(
229     version: int,
230 ) -> Callable[
231     [Callable[[aiosqlite.Connection], Awaitable[None]]],
232     Callable[[aiosqlite.Connection], Awaitable[None]],
233 ]:
234     """Register a migration that brings the DB up to ``user_version == version``."""
235
236     def decorator(
237         fn: Callable[[aiosqlite.Connection], Awaitable[None]],
238     ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
239         _MIGRATIONS.append((version, fn))
240         return fn
241
242     return decorator
243
244
245 # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
246 #
247 # Pure function: computes the ``content_items.dedup_key`` value from metadata
248 # already extracted at ingestion time (no download ? honors ADR 0003). The
249 # decision tree is ordered strongest-key first; each branch is guarded so
250 # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
251 # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
252
253
254 def compute_dedup_key(
255     *,
256     provider: str,
257     document_id: str | None = None,
258     source_id: int | str | None = None,
259     provider_message_id: str | None = None,
260     size_bytes: int | None = None,
261     duration_s: int | None = None,
262     height: int | None = None,

```

```

263         width: int | None = None,
264     ) -> str:
265         """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
266
267         Implements the ADR 0011 Option 1A decision tree:
268
269         - **Telegram** (``provider == "telegram"``):
270             1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
271             2. Else if **all four** fingerprint fields (size, duration, height, width)
272                are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273             3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274                ``source_id`` is required because Telegram ``message_id`` is only
275                unique within a channel).
276         - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
277           ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278           providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279
280         Raises ``ValueError`` if the fallback branch is reached without the fields
281         it requires (``source_id`` + ``provider_message_id``) ? a degenerate
282         ``dedup_key`` is worse than a loud failure at ingestion time.
283         """
284         p = provider.lower()
285         if p == "telegram":
286             if document_id is not None and document_id != "":
287                 return f"tg:doc:{document_id}"
288             # All four fingerprint fields are required to avoid a degenerate
289             # ``tg:fp:0:0:0:0`` key when some metadata is missing.
290             if None not in (size_bytes, duration_s, height, width):
291                 return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292             # Tier-0 fallback.
293             if source_id is None or provider_message_id is None:
294                 raise ValueError(
295                     "Telegram dedup_key fallback requires source_id and provider_message_id"
296                 )
297             return f"tg:msg:{source_id}:{provider_message_id}"
298             # Future-proof default for providers without a dedicated branch.
299             if source_id is None or provider_message_id is None:
300                 raise ValueError(
301                     f"dedup_key for provider={provider!r} requires source_id and"
302                     f"provider_message_id"
303                 )
304             return f"{p}:msg:{source_id}:{provider_message_id}"
305
306
307     async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308         """Run every registered migration whose version > current ``user_version``."""
309         async with conn.execute("PRAGMA user_version") as cur:
310             row = await cur.fetchone()
311             current = int(row[0]) if row else 0
312
313         for version, fn in _MIGRATIONS:
314             if version <= current:
315                 continue
316             await fn(conn)
317             await conn.execute(f"PRAGMA user_version = {version}")
318
319
320     async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
321         """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
322         async with conn.execute(f"PRAGMA table_info({table})") as cur:
323             rows = await cur.fetchall()
324             return any(row[1] == column for row in rows)
325

```

```

326
327 @_migration(1)
328 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
329     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
330
331     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
332     created before this migration existed.
333     """
334     if not await _column_exists(conn, "videos", "document_id"):
335         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
336     # Create the partial unique index for both fresh and migrated DBs. Uses
337     # IF NOT EXISTS so it is safe on DBs where it was already created.
338     await conn.execute(
339         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
340         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
341     )
342
343
344 @_migration(2)
345 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
346     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
347
348     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
349     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
350     so their integer ``user_id`` values still resolve to the same identity (now
351     via ``accounts.id``). The legacy ``users`` table is renamed to
352     ``users_deprecated`` and kept for one release as a rollback safety net; a
353     follow-up migration drops it.
354
355     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
356     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
357     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
358     those FKs to point at ``users_deprecated``, diverging from fresh installs
359     and breaking inserts for new accounts ? see PR #33 review.)
360
361     Idempotent: a no-op once the legacy ``users`` table has been renamed.
362     """
363     # If the legacy users table is gone, this migration has already run
364     # (or the DB is fresh and only has accounts). Nothing to do.
365     if not await _table_exists(conn, "users"):
366         return
367
368     # Build accounts from the legacy users table. accounts.id := users.id.
369     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
370     # and recreated here so the INSERT can populate it cleanly.)
371     await conn.execute("DROP TABLE IF EXISTS accounts")
372     await conn.execute(
373         """
374         CREATE TABLE accounts (
375             id            INTEGER PRIMARY KEY,
376             telegram_id   INTEGER UNIQUE,
377             provider      TEXT NOT NULL DEFAULT 'telegram',
378             display_name  TEXT,
379             created_at    TEXT NOT NULL
380         )
381         """
382     )
383     await conn.execute(
384         """
385         INSERT INTO accounts (id, telegram_id, provider, created_at)
386         SELECT id, id, 'telegram', created_at FROM users
387         """
388     )
389
390     # Build telegram_sessions, one row per legacy userbot user.

```

```

391 # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
392 await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
393 await conn.execute(
394     """
395     CREATE TABLE telegram_sessions (
396         account_id INTEGER PRIMARY KEY REFERENCES accounts(id),
397         phone TEXT,
398         session_path TEXT NOT NULL,
399         enabled INTEGER NOT NULL DEFAULT 1,
400         created_at TEXT NOT NULL,
401         last_login_at TEXT
402     )
403     """
404 )
405 # session_path is built from the *telegram_id* (= old users.id) to match
406 # the convention telethon_client.build_client used pre-migration. The
407 # sessions_dir prefix is prepended at read time (Settings.sessions_dir)
408 # so this migration does not need to know the runtime path.
409 await conn.execute(
410     """
411     INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
412     SELECT id, phone, 'user_' || id || '.session', enabled, created_at
413     FROM users
414     """
415 )
416
417 # Rebuild channels / videos / filter_config so their FK constraints point
418 # at accounts(id) instead of the legacy users(id). SQLite's
419 # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
420 # the rename ? so a naive rename of users ? users_deprecated would leave
421 # these tables referencing users_deprecated(id), diverging from fresh DBs
422 # (which reference accounts(id)) and breaking inserts for new accounts.
423 # The rebuild: rename the data-holding legacy table out of the way, create
424 # the new one with the correct FK, copy the data back, drop the renamed copy.
425 for table, new_ddl, copy_cols in (
426     (
427         "channels",
428         """
429         CREATE TABLE channels (
430             id INTEGER PRIMARY KEY AUTOINCREMENT,
431             user_id INTEGER NOT NULL REFERENCES accounts(id),
432             channel_id INTEGER NOT NULL,
433             title TEXT,
434             seen_first_at TEXT NOT NULL,
435             UNIQUE (user_id, channel_id)
436         )
437         """,
438         "id, user_id, channel_id, title, seen_first_at",
439     ),
440     (
441         "videos",
442         """
443         CREATE TABLE videos (
444             id INTEGER PRIMARY KEY AUTOINCREMENT,
445             user_id INTEGER NOT NULL REFERENCES accounts(id),
446             channel_id INTEGER NOT NULL,
447             message_id INTEGER NOT NULL,
448             document_id INTEGER,
449             duration_s INTEGER,
450             width INTEGER,
451             height INTEGER,
452             size_bytes INTEGER,
453             mime_type TEXT,
454             matched INTEGER NOT NULL DEFAULT 0,

```

```

455         forwarded    INTEGER NOT NULL DEFAULT 0,
456         seen_at      TEXT NOT NULL,
457         UNIQUE (user_id, channel_id, message_id)
458     )
459     """
460     "id, user_id, channel_id, message_id, document_id, duration_s, width, "
461     "height, size_bytes, mime_type, matched, forwarded, seen_at",
462 ),
463 (
464     "filter_config",
465     """
466     CREATE TABLE filter_config (
467         user_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
468         config_json TEXT NOT NULL
469     )
470     """
471     "user_id, config_json",
472 ),
473 ):
474     await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
475     await conn.execute(new_ddl)
476     await conn.execute(
477         f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
478     )
479     await conn.execute(f"DROP TABLE {table}__legacy_data")
480
481     # The partial unique index for document_id dedup (created by migration 001
482     # against the legacy videos table) was dropped along with the legacy copy
483     # above. Recreate it on the new videos table.
484     await conn.execute(
485         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
486         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
487     )
488
489     # Rename the legacy users table (do NOT drop ? kept as a rollback safety
490     # net for one release per ADR 0010). The data-holding channels/videos/
491     # filter_config tables were already rebuilt above with FKs to accounts(id),
492     # so renaming users no longer retargets any live FK.
493     await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
494
495
496 @_migration(3)
497 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498     """Land the provider-neutral domain tables (ADR 0011, Track B).
499
500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     # channels stay watched (watched=1). provider_source_id is the Telegram
516     # channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """

```



```

519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521              seen_first_at, last_seen_at)
522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523             1, seen_first_at, seen_first_at
524         FROM channels
525         """"
526     )
527
528 # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529 #     The three-branch decision tree (Option 1A):
530 #         a) document_id IS NOT NULL           ? 'tg:doc:' || document_id
531 #         b) all four fingerprint fields set   ? 'tg:fp:' || size:dur:h:w
532 #         c) else (Tier-0 fallback)            ? 'tg:msg:' || source_id:msg_id
533 #     The ``CASE`` is ordered strongest-first and each branch is guarded so
534 #     partial metadata never collapses to a degenerate key. source_id is
535 #     resolved by joining the channel's provider_source_id back to sources.id
536 #     (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """"
539     INSERT OR IGNORE INTO content_items
540         (account_id, source_id, provider, provider_message_id, content_type,
541          document_id, duration_s, width, height, size_bytes, mime_type,
542          dedup_key, matched, seen_at)
543     SELECT
544         v.user_id,
545         s.id,
546         'telegram',
547         CAST(v.message_id AS TEXT),
548         'video',
549         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551         CASE
552             WHEN v.document_id IS NOT NULL
553             THEN 'tg:doc:' || v.document_id
554             WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555                 AND v.height IS NOT NULL AND v.width IS NOT NULL
556             THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557                 || v.height || ':' || v.width
558             ELSE 'tg:msg:' || s.id || ':' || v.message_id
559         END,
560         v.matched,
561         v.seen_at
562     FROM videos v
563     JOIN sources s
564         ON s.account_id = v.user_id
565         AND s.provider = 'telegram'
566         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """"
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """"
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576                                     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """"
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 #     rollback safety net (ADR 0010 pattern).

```

```

584     await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585     await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586     await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591         async with conn.execute(
592             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593         ) as cur:
594             return await cur.fetchone() is not None
595
596
597     # --- helpers -----
598
599
600     def _now() -> datetime:
601         """Timezone-aware UTC 'now', used for policy timestamps."""
602         return datetime.now(UTC)
603
604
605     def _row_to_account(row: aiosqlite.Row) -> Account:
606         return Account(
607             id=row["id"],
608             telegram_id=row["telegram_id"],
609             provider=row["provider"],
610             display_name=row["display_name"],
611             created_at=row["created_at"], # stored as ISO string; pydantic parses
612         )
613
614
615     def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616         return TelegramSession(
617             account_id=row["account_id"],
618             phone=row["phone"],
619             session_path=row["session_path"],
620             enabled=bool(row["enabled"]),
621             created_at=row["created_at"],
622             last_login_at=row["last_login_at"],
623         )
624
625
626     def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
627         """Project an (account, optional session) pair onto the legacy ``User`` view.
628
629         Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
630         shims so pre-ADR-0010 callers and tests keep working. A bot-only account
631         (no session) maps to a ``User`` with ``enabled=False`` so the legacy
632         "list enabled users" semantics exclude it ? matching the structural rule
633         that bot-only users are not runnable Telethon clients.
634         """
635         if session is None:
636             return User(
637                 id=account.telegram_id if account.telegram_id is not None else account.id,
638                 phone=None,
639                 enabled=False,
640                 created_at=account.created_at,
641             )
642         return User(
643             id=account.telegram_id if account.telegram_id is not None else account.id,
644             phone=session.phone,
645             enabled=session.enabled,
646             created_at=session.created_at,
647         )
648

```

```

649
650 def _row_to_channel(row: aiosqlite.Row) -> Channel:
651     return Channel(
652         id=row["id"],
653         user_id=row["user_id"],
654         channel_id=row["channel_id"],
655         title=row["title"],
656         seen_first_at=row["seen_first_at"],
657     )
658
659
660 def _row_to_video(row: aiosqlite.Row) -> VideoRecord:
661     return VideoRecord(
662         id=row["id"],
663         user_id=row["user_id"],
664         channel_id=row["channel_id"],
665         message_id=row["message_id"],
666         document_id=row["document_id"],
667         duration_s=row["duration_s"],
668         width=row["width"],
669         height=row["height"],
670         size_bytes=row["size_bytes"],
671         mime_type=row["mime_type"],
672         matched=bool(row["matched"]),
673         forwarded=bool(row["forwarded"]),
674         seen_at=row["seen_at"],
675     )
676
677
678 def _row_to_source(row: aiosqlite.Row) -> Source:
679     return Source(
680         id=row["id"],
681         account_id=row["account_id"],
682         provider=row["provider"],
683         provider_source_id=row["provider_source_id"],
684         title=row["title"],
685         kind=row["kind"],
686         watched=bool(row["watched"]),
687         seen_first_at=row["seen_first_at"],
688         last_seen_at=row["last_seen_at"],
689     )
690
691
692 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
693     return ContentItem(
694         id=row["id"],
695         account_id=row["account_id"],
696         source_id=row["source_id"],
697         provider=row["provider"],
698         provider_message_id=row["provider_message_id"],
699         content_type=row["content_type"],
700         text=row["text"],
701         link_url=row["link_url"],
702         media_ref=row["media_ref"],
703         document_id=row["document_id"],
704         duration_s=row["duration_s"],
705         width=row["width"],
706         height=row["height"],
707         size_bytes=row["size_bytes"],
708         mime_type=row["mime_type"],
709         dedup_key=row["dedup_key"],
710         matched=bool(row["matched"]),
711         seen_at=row["seen_at"],
712     )
713

```

```

714
715 def _row_to_policy(row: aiosqlite.Row) -> Policy:
716     return Policy(
717         id=row["id"],
718         account_id=row["account_id"],
719         name=row["name"],
720         config_json=row["config_json"],
721         is_default=bool(row["is_default"]),
722         created_at=row["created_at"],
723         updated_at=row["updated_at"],
724     )
725
726
727 # --- account + session repos (ADR 0010) -----
728
729
730 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
731     """Insert or update an account row; returns the persisted Account."""
732     await conn.execute(
733         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734         VALUES (?, ?, ?, ?, ?)
735         ON CONFLICT(id) DO UPDATE SET
736             telegram_id=excluded.telegram_id,
737             provider=excluded.provider,
738             display_name=excluded.display_name""",
739         (
740             account.id,
741             account.telegram_id,
742             account.provider,
743             account.display_name,
744             account.created_at.isoformat(),
745         ),
746     )
747     await conn.commit()
748     return await get_account(conn, account.id) # type: ignore[return-value]
749
750
751 async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
752     conn.row_factory = aiosqlite.Row
753     async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as cur:
754         row = await cur.fetchone()
755         return _row_to_account(row) if row else None
756
757
758 async def get_account_by_telegram_id(
759     conn: aiosqlite.Connection, telegram_id: int
760 ) -> Account | None:
761     """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
762     conn.row_factory = aiosqlite.Row
763     async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
764 (telegram_id,)) as cur:
765         row = await cur.fetchone()
766         return _row_to_account(row) if row else None
767
768
769 async def upsert_telegram_session(
770     conn: aiosqlite.Connection, session: TelegramSession
771 ) -> TelegramSession:
772     """Insert or update a telegram_sessions row; returns the persisted session."""
773     await conn.execute(
774         """INSERT INTO telegram_sessions
775         (account_id, phone, session_path, enabled, created_at, last_login_at)
776         VALUES (?, ?, ?, ?, ?, ?)
777         ON CONFLICT(account_id) DO UPDATE SET

```

```

777         phone=excluded.phone,
778         session_path=excluded.session_path,
779         enabled=excluded.enabled,
780         last_login_at=excluded.last_login_at"",
781     (
782         session.account_id,
783         session.phone,
784         session.session_path,
785         int(session.enabled),
786         session.created_at.isoformat(),
787         session.last_login_at.isoformat() if session.last_login_at else None,
788     ),
789 )
790 await conn.commit()
791 conn.row_factory = aiomysqlite.Row
792 async with conn.execute(
793     "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)
794 ) as cur:
795     row = await cur.fetchone()
796     # The row must exist ? we just upserted it by PK (account_id). Assert the
797     # invariant explicitly so mypy can narrow Row | None ? Row without a
798     # type-ignore comment masking a real future regression.
799     assert row is not None
800     return _row_to_telegram_session(row)
801
802
803 async def get_telegram_session(
804     conn: aiomysqlite.Connection, account_id: int
805 ) -> TelegramSession | None:
806     conn.row_factory = aiomysqlite.Row
807     async with conn.execute(
808         "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
809     ) as cur:
810         row = await cur.fetchone()
811         return _row_to_telegram_session(row) if row else None
812
813
814 async def list_runnable_sessions(
815     conn: aiomysqlite.Connection,
816 ) -> list[tuple[Account, TelegramSession]]:
817     """Return (account, session) pairs for sessions the manager should start.
818
819     This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
820     not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
821     therefore inherently invisible to the Telethon supervisor ? the "don't
822     start a client for bot-only users" invariant is enforced by query shape.
823     """
824     conn.row_factory = aiomysqlite.Row
825     async with conn.execute(
826         """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
827             s.created_at AS s_created_at, s.last_login_at
828             FROM accounts a
829             JOIN telegram_sessions s ON s.account_id = a.id
830             WHERE s.enabled = 1"""
831     ) as cur:
832         rows = await cur.fetchall()
833     return [
834         (
835             _row_to_account(r),
836             TelegramSession(
837                 account_id=r["id"],
838                 phone=r["s_phone"],
839                 session_path=r["session_path"],
840                 enabled=bool(r["s_enabled"]),
841                 created_at=r["s_created_at"],

```

```

842         last_login_at=r["last_login_at"],
843     ),
844 )
845     for r in rows
846 ]
847
848
849 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
850
851
852 async def upsert_user(conn: aiomysqlite.Connection, user: User) -> User:
853     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
854
855     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
856     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
857     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
858     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
859     """
860     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
861     await upsert_account(conn, account)
862     await upsert_telegram_session(
863         conn,
864         TelegramSession(
865             account_id=user.id,
866             phone=user.phone,
867             session_path=f"user_{user.id}.session",
868             enabled=user.enabled,
869             created_at=user.created_at,
870         ),
871     )
872     return await get_user(conn, user.id) # type: ignore[return-value]
873
874
875 async def get_user(conn: aiomysqlite.Connection, user_id: int) -> User | None:
876     """Deprecated shim: project the (account, session) pair back to a ``User``."""
877     account = await get_account(conn, user_id)
878     if account is None:
879         return None
880     session = await get_telegram_session(conn, user_id)
881     return _account_session_to_user(account, session)
882
883
884 async def list_enabled_users(conn: aiomysqlite.Connection) -> list[User]:
885     """Deprecated shim: list users that have an enabled MTPROTO session.
886
887     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
888     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
889     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
890     """
891     pairs = await list_runnable_sessions(conn)
892     return [_account_session_to_user(a, s) for a, s in pairs]
893
894
895 # --- Source repos (ADR 0011) -----
896
897
898 async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:
899     """Insert a source row, or update title/watched on conflict.
900
901     The conflict key is ``(account_id, provider, provider_source_id)`` per the
902     ADR 0011 schema. Returns the persisted row.
903     """
904     await conn.execute(
905         """INSERT INTO sources
906             (account_id, provider, provider_source_id, title, kind, watched,

```

```

907         seen_first_at, last_seen_at)
908     VALUES (?, ?, ?, ?, ?, ?, ?, ?)
909     ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
910         title=excluded.title,
911         watched=excluded.watched,
912         last_seen_at=excluded.last_seen_at""",
913     (
914         source.account_id,
915         source.provider,
916         source.provider_source_id,
917         source.title,
918         source.kind,
919         int(source.watched),
920         source.seen_first_at.isoformat(),
921         (source.last_seen_at.isoformat() if source.last_seen_at else None),
922     ),
923 )
924 await conn.commit()
925 conn.row_factory = aiosqlite.Row
926 async with conn.execute(
927     """SELECT * FROM sources
928     WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
929     (source.account_id, source.provider, source.provider_source_id),
930 ) as cur:
931     row = await cur.fetchone()
932     assert row is not None # just upserted by the conflict key
933     return _row_to_source(row)
934
935
936 async def get_source_by_provider_id(
937     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
938 ) -> Source | None:
939     """Resolve a source by its provider-native id."""
940     conn.row_factory = aiosqlite.Row
941     async with conn.execute(
942         """SELECT * FROM sources
943         WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
944         (account_id, provider, provider_source_id),
945     ) as cur:
946         row = await cur.fetchone()
947         return _row_to_source(row) if row else None
948
949
950 # --- ContentItem repos (ADR 0011) -----
951
952
953 async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
954     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
955
956     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
957     unique constraints guard against duplicates:
958     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
959     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
960     """
961     cur = await conn.execute(
962         """INSERT INTO content_items
963         (account_id, source_id, provider, provider_message_id, content_type,
964         text, link_url, media_ref, document_id, duration_s, width, height,
965         size_bytes, mime_type, dedup_key, matched, seen_at)
966         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
967         (
968             item.account_id,
969             item.source_id,
970             item.provider,

```

```

971         item.provider_message_id,
972         item.content_type,
973         item.text,
974         item.link_url,
975         item.media_ref,
976         item.document_id,
977         item.duration_s,
978         item.width,
979         item.height,
980         item.size_bytes,
981         item.mime_type,
982         item.dedup_key,
983         int(item.matched),
984         item.seen_at.isoformat(),
985     ),
986 )
987 await conn.commit()
988 return item.model_copy(update={"id": cur.lastrowid})
989
990
991 async def get_content_item_by_message(
992     conn: aiosqlite.Connection,
993     account_id: int,
994     source_id: int,
995     provider_message_id: str,
996 ) -> ContentItem | None:
997     """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
998     conn.row_factory = aiosqlite.Row
999     async with conn.execute(
1000         """SELECT * FROM content_items
1001            WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1002         (account_id, source_id, provider_message_id),
1003     ) as cur:
1004         row = await cur.fetchone()
1005     return _row_to_content_item(row) if row else None
1006
1007
1008 async def get_content_item_by_dedup_key(
1009     conn: aiosqlite.Connection, account_id: int, dedup_key: str
1010 ) -> ContentItem | None:
1011     """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1012
1013     Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014     a single query: the ``dedup_key`` already encodes whichever tier was
1015     applicable at ingestion time.
1016
1017     """
1018     conn.row_factory = aiosqlite.Row
1019     async with conn.execute(
1020         "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1021         (account_id, dedup_key),
1022     ) as cur:
1023         row = await cur.fetchone()
1024     return _row_to_content_item(row) if row else None
1025
1026 # --- Policy repos (ADR 0011, Option 2A) -----
1027
1028
1029 async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1030     """Insert or update a policy row by ``(account_id, name)``.
1031
1032     Exactly one policy per account should carry ``is_default=True``; callers
1033     are responsible for flipping the previous default before flipping a new
1034     one (enforced by a partial unique index landed by future work, not in the
1035     schema ? SQLite ``CHECK`` can't express "at most one per group").

```



```

1036     """
1037     await conn.execute(
1038         """INSERT INTO policies (account_id, name, config_json, is_default,
1039             created_at, updated_at)
1040             VALUES (?, ?, ?, ?, ?, ?)
1041             ON CONFLICT(account_id, name) DO UPDATE SET
1042                 config_json=excluded.config_json,
1043                 is_default=excluded.is_default,
1044                 updated_at=excluded.updated_at""",
1045         (
1046             policy.account_id,
1047             policy.name,
1048             policy.config_json,
1049             int(policy.is_default),
1050             policy.created_at.isoformat(),
1051             policy.updated_at.isoformat(),
1052         ),
1053     )
1054     await conn.commit()
1055     conn.row_factory = aiosqlite.Row
1056     async with conn.execute(
1057         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1058         (policy.account_id, policy.name),
1059     ) as cur:
1060         row = await cur.fetchone()
1061         assert row is not None
1062         return _row_to_policy(row)
1063
1064
1065     async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1066         """Return the account's default policy (`is_default=1`)."""
1067         conn.row_factory = aiosqlite.Row
1068         async with conn.execute(
1069             "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1070             (account_id,),
1071         ) as cur:
1072             row = await cur.fetchone()
1073             return _row_to_policy(row) if row else None
1074
1075
1076     # --- legacy Channel shim (backwards-compatible view over sources) -----
1077
1078
1079     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1080         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1081
1082         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1083         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1084         New code should call :func:`upsert_source` directly.
1085         """
1086         src = await upsert_source(
1087             conn,
1088             Source(
1089                 account_id=channel.user_id,
1090                 provider="telegram",
1091                 provider_source_id=str(channel.channel_id),
1092                 title=channel.title,
1093                 kind="channel",
1094                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1095                 seen_first_at=channel.seen_first_at,
1096             ),
1097         )
1098         return _source_to_channel(src)
1099

```

```

1100
1101     async def get_channel(conn: aioredis.Connection, user_id: int, channel_id: int) ->
Channel | None:
1102         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1103         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1104         return _source_to_channel(src) if src else None
1105
1106
1107     def _source_to_channel(src: Source) -> Channel:
1108         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
1109         return Channel(
1110             id=src.id,
1111             user_id=src.account_id,
1112             channel_id=int(src.provider_source_id),
1113             title=src.title,
1114             seen_first_at=src.seen_first_at,
1115         )
1116
1117
1118     # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1119
1120
1121     async def get_video(
1122         conn: aioredis.Connection, user_id: int, channel_id: int, message_id: int
1123     ) -> VideoRecord | None:
1124         """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126         Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127         :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128         :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129         """
1130         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131         if src is None:
1132             return None
1133         item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134         return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137     async def get_video_by_document_id(
1138         conn: aioredis.Connection, user_id: int, document_id: int
1139     ) -> VideoRecord | None:
1140         """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141         dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143         if item is None:
1144             return None
1145         src = await get_source_by_id(conn, item.source_id)
1146         if src is None:
1147             return None
1148         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151     async def get_video_by_content_fingerprint(
1152         conn: aioredis.Connection,
1153         user_id: int,
1154         size_bytes: int,
1155         duration_s: int,
1156         height: int,
1157         width: int,
1158     ) -> VideoRecord | None:
1159         """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160         dedup_key = compute_dedup_key(
1161             provider="telegram",
1162             size_bytes=size_bytes,
1163             duration_s=duration_s,

```

```

1164         height=height,
1165         width=width,
1166     )
1167     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168     if item is None:
1169         return None
1170     src = await get_source_by_id(conn, item.source_id)
1171     if src is None:
1172         return None
1173     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176 async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) -> Source | None:
1177     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178     conn.row_factory = aiomysqlite.Row
1179     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180         row = await cur.fetchone()
1181     return _row_to_source(row) if row else None
1182
1183
1184 async def insert_video(conn: aiomysqlite.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),
1201     )
1202     assert src.id is not None # upsert_source populates the surrogate PK
1203     item = ContentItem(
1204         account_id=video.user_id,
1205         source_id=src.id,
1206         provider="telegram",
1207         provider_message_id=str(video.message_id),
1208         content_type="video",
1209         document_id=str(video.document_id) if video.document_id is not None else None,
1210         duration_s=video.duration_s,
1211         width=video.width,
1212         height=video.height,
1213         size_bytes=video.size_bytes,
1214         mime_type=video.mime_type,
1215         dedup_key=compute_dedup_key(
1216             provider="telegram",
1217             document_id=(str(video.document_id) if video.document_id is not None else
1218 None),
1219             source_id=src.id,
1220             provider_message_id=str(video.message_id),
1221             size_bytes=video.size_bytes,
1222             duration_s=video.duration_s,
1223             height=video.height,
1224             width=video.width,
1225         ),
1226         matched=video.matched,
1227         seen_at=video.seen_at,

```

```

1228     item = await insert_content_item(conn, item)
1229     return _content_item_to_video(item, src, video.user_id, video.channel_id)
1230
1231
1232 async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1233     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238     delivery still targets the single ``delivery_target`` and retried delivery
1239     is harmless (forwards are idempotent). This shim is retained so existing
1240     call sites in ``delivery.py`` keep working without a flag day.
1241     """
1242     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1243     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1244     return None
1245
1246
1247 def _content_item_to_video(
1248     item: ContentItem, src: Source, user_id: int, channel_id: int
1249 ) -> VideoRecord:
1250     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1253     per-destination delivery state moves to Track C). Existing callers read it
1254     only to decide whether to skip re-delivery; since forwards are idempotent
1255     the ``False`` default is safe for Phase 1.
1256     """
1257     return VideoRecord(
1258         id=item.id,
1259         user_id=user_id,
1260         channel_id=channel_id,
1261         message_id=int(item.provider_message_id),
1262         document_id=int(item.document_id) if item.document_id is not None else None,
1263         duration_s=item.duration_s,
1264         width=item.width,
1265         height=item.height,
1266         size_bytes=item.size_bytes,
1267         mime_type=item.mime_type,
1268         matched=item.matched,
1269         forwarded=False,
1270         seen_at=item.seen_at,
1271     )
1272
1273
1274 # --- filter config shims (backwards-compatible view over policies) -----
1275
1276
1277 async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1278     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1279
1280     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1281     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1282     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1283     """
1284     policy = await get_default_policy(conn, user_id)
1285     if policy is None:
1286         return FilterConfig()
1287     return FilterConfig.model_validate_json(policy.config_json)
1288
1289
1290 async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1291     """Deprecated shim: upsert the account's default :class:`Policy`."""

```

```

1292     existing = await get_default_policy(conn, user_id)
1293     await upsert_policy(
1294         conn,
1295         Policy(
1296             id=existing.id if existing is not None else None,
1297             account_id=user_id,
1298             name="default",
1299             config_json=cfg.model_dump_json(),
1300             is_default=True,
1301             created_at=(existing.created_at if existing is not None else _now()),
1302             updated_at=_now(),
1303         ),
1304     )
1305
1306
1307 # --- bulk helpers (used by the manager on startup) -----
1308
1309
1310 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1311     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aiosqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid
1317            FROM content_items ci
1318            JOIN sources s ON s.id = ci.source_id
1319            WHERE ci.account_id = ?
1320            ORDER BY ci.id DESC LIMIT ?""",
1321         (user_id, limit),
1322     ) as cur:
1323         rows = await cur.fetchall()
1324     return [
1325         VideoRecord(
1326             id=r["id"],
1327             user_id=user_id,
1328             channel_id=int(r["src_pid"]),
1329             message_id=int(r["provider_message_id"]),
1330             document_id=(int(r["document_id"]) if r["document_id"] is not None else
1331 None),
1332             duration_s=r["duration_s"],
1333             width=r["width"],
1334             height=r["height"],
1335             size_bytes=r["size_bytes"],
1336             mime_type=r["mime_type"],
1337             matched=bool(r["matched"]),
1338             forwarded=False,
1339             seen_at=r["seen_at"],
1340         )
1341         for r in rows
1342     ]
1343
1344 __all__ = [
1345     "init_db",
1346     "compute_dedup_key",
1347     # Account + session repos (ADR 0010)
1348     "upsert_account",
1349     "get_account",
1350     "get_account_by_telegram_id",
1351     "upsert_telegram_session",
1352     "get_telegram_session",
1353     "list_runnable_sessions",
1354     # Provider-neutral domain repos (ADR 0011)
1355     "upsert_source",

```

```

1356     "get_source_by_provider_id",
1357     "get_source_by_id",
1358     "insert_content_item",
1359     "get_content_item_by_message",
1360     "get_content_item_by_dedup_key",
1361     "upsert_policy",
1362     "get_default_policy",
1363     # Legacy User shims (backwards-compatible)
1364     "upsert_user",
1365     "get_user",
1366     "list_enabled_users",
1367     # Legacy Channel / Video / config shims (ADR 0011)
1368     "upsert_channel",
1369     "get_channel",
1370     "get_video",
1371     "get_video_by_document_id",
1372     "get_video_by_content_fingerprint",
1373     "insert_video",
1374     "mark_forwarded",
1375     "load_filter_config",
1376     "save_filter_config",
1377     "list_videos_for_user",
1378 ]
1379
1380
1381 def __dir__() -> Iterable[str]:
1382     return __all__
1383

```

5. assistant (2026-07-06T06:02:59.256Z)

Now let me look at the v1 behaviour on main and the test files.

6. user (2026-07-06T06:03:04.405Z)

"""SQLite persistence layer (aiosqlite).

Schema, migrations, and repository functions for users, channels, videos, and filter_config. See ``docs/design.md`` §3 for the data model.

The repository functions take an ``aiosqlite.Connection`` so tests can inject an in-memory database (``aiosqlite.connect(":memory:")``).

```
from __future__ import annotations
```

```
from collections.abc import Awaitable, Callable, Iterable
```

```
import aiosqlite
```

```

from tg_video_filter.models import (
    Account,
    Channel,
    FilterConfig,
    TelegramSession,
    User,
    VideoRecord,
)

```

--- DDL -----

```

_SCHEMA = """
CREATE TABLE IF NOT EXISTS accounts (
    id            INTEGER PRIMARY KEY,

```

```

        telegram_id INTEGER UNIQUE,
        provider     TEXT NOT NULL DEFAULT 'telegram',
        display_name TEXT,
        created_at   TEXT NOT NULL
    );

```

```

CREATE TABLE IF NOT EXISTS telegram_sessions (
    account_id INTEGER PRIMARY KEY REFERENCES accounts(id),
    phone       TEXT,
    session_path TEXT NOT NULL,
    enabled     INTEGER NOT NULL DEFAULT 1,
    created_at  TEXT NOT NULL,
    last_login_at TEXT
);

```

```

CREATE TABLE IF NOT EXISTS channels (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL REFERENCES accounts(id),
    channel_id    INTEGER NOT NULL,
    title         TEXT,
    seen_first_at TEXT NOT NULL,

```

===GREP===

```

608:async def get_video(
622:async def get_video_by_document_id(
640:async def get_video_by_content_fingerprint(
765:     "get_video_by_document_id",
766:     "get_video_by_content_fingerprint",

```

7. user (2026-07-06T06:03:04.538Z)

```

src\tg_video_filter\backfill.py:171:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\backfill.py:192:             existing = await
db.get_video_by_content_fingerprint(
src\tg_video_filter\db.py:1137:async def get_video_by_document_id(
src\tg_video_filter\db.py:1151:async def get_video_by_content_fingerprint(
src\tg_video_filter\db.py:1371:     "get_video_by_document_id",
src\tg_video_filter\db.py:1372:     "get_video_by_content_fingerprint",
src\tg_video_filter\telethon_client.py:193:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\telethon_client.py:214:             existing = await
db.get_video_by_content_fingerprint(

```

8. assistant (2026-07-06T06:03:06.953Z)

Let me see the v1 fingerprint query and the caller logic in telethon_client.py.

9. user (2026-07-06T06:03:11.468Z)

--- video repos -----

```

async def get_video(
    conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
) -> VideoRecord | None:
    """Look up a video by its dedup key. Returns None if not seen before."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ? AND channel_id = ? AND message_id = ?""",
        (user_id, channel_id, message_id),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None

```

```

async def get_video_by_document_id(
    conn: aiosqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:
    """Tier-1 content dedup: same Telegram file id ? same video.

    Telegram reuses ``document.id`` when the same file is forwarded across
    channels, so this catches cross-channel duplicates at zero cost.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ? AND document_id = ?""",
        (user_id, document_id),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None


async def get_video_by_content_fingerprint(
    conn: aiosqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: metadata fingerprint match ? probably same video.

    Catches re-uploads where Telegram assigns a new ``document.id`` but the
    file bytes are identical. Uses (size_bytes, duration_s, height, width)
    as a composite key ? the odds of two different videos sharing all four
    values are negligible for this use case.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ?
             AND size_bytes = ?
             AND duration_s = ?
             AND height = ?
             AND width = ?
           LIMIT 1""",
        (user_id, size_bytes, duration_s, height, width),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None


async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Insert a new video row. Raises IntegrityError on a dedup-key collision.

    Two dedup constraints protect against duplicates:
    - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
    - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
      same file forwarded across channels (content dedup Tier 1).
    """
    cur = await conn.execute(
        """INSERT INTO videos
           (user_id, channel_id, message_id, document_id, duration_s, width, height,
            size_bytes, mime_type, matched, forwarded, seen_at)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            video.user_id,
            video.channel_id,
            video.message_id,

```



```
video.document_id,  
video.duration_s,  
video.width,  
video.height,
```

10. user (2026-07-06T06:03:11.473Z)

```
175  
176     meta = extract_video_meta(message)  
177     if meta is None:  
178         return # not a video  
179  
180     channel_id = _peer_id(message.peer_id)  
181     message_id = message.id  
182     document_id = _extract_document_id(message)  
183  
184     # --- three-tier dedup (cheapest first) ---  
185  
186     # Tier 0: same (user, channel, message) instance ? already processed.  
187     existing = await db.get_video(conn, user.id, channel_id, message_id)  
188     if existing is not None:  
189         return  
190  
191     # Tier 1: same Telegram document.id ? forwarded/cross-posted file.  
192     if document_id is not None:  
193         existing = await db.get_video_by_document_id(conn, user.id, document_id)  
194         if existing is not None:  
195             log.info(  
196                 "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "  
197                 "original_channel=%s original_msg=%s",  
198                 user.id,  
199                 document_id,  
200                 channel_id,  
201                 message_id,  
202                 existing.channel_id,  
203                 existing.message_id,  
204             )  
205             return  
206  
207     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).  
208     if (  
209         meta.size_bytes is not None  
210         and meta.duration_s is not None  
211         and meta.height is not None  
212         and meta.width is not None  
213     ):  
214         existing = await db.get_video_by_content_fingerprint(  
215             conn,  
216             user.id,  
217             meta.size_bytes,  
218             meta.duration_s,  
219             meta.height,  
220             meta.width,  
221         )  
222         if existing is not None:  
223             log.info(  
224                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "  
225                 "channel=%s msg=%s original_channel=%s original_msg=%s",  
226                 user.id,  
227                 meta.size_bytes,  
228                 meta.duration_s,  
229                 meta.height,  
230                 meta.width,  
231                 channel_id,  
232                 message_id,
```

```

233             existing.channel_id,
234             existing.message_id,
235         )
236         return
237
238     # Lazily record the channel (best-effort; failures are non-fatal).
239     try:
240         await db.upsert_channel(
241             conn,
242             Channel(user_id=user.id, channel_id=channel_id,
243 title=_channel_title(event)),
244         )
245     except Exception:
246         log.warning(
247             "failed to upsert channel %s for user %s", channel_id, user.id,
248 exc_info=True
249         )
250
251     result = engine_holder[ENGINE_KEY].evaluate(meta)
252     record = VideoRecord(
253         user_id=user.id,
254         channel_id=channel_id,
255         message_id=message_id,
256         document_id=document_id,
257         duration_s=meta.duration_s,
258         width=meta.width,
259         height=meta.height,
260         size_bytes=meta.size_bytes,
261         mime_type=meta.mime_type,
262         matched=result.matched,
263     )
264     try:
265         inserted = await db.insert_video(conn, record)
266     except sqlite3.IntegrityError:

```

11. assistant (2026-07-06T06:03:15.885Z)

Now I have the full picture. Let me confirm the seed claim with a repro. The key insight: v1 Tier-2 query is `SELECT ... WHERE size/dur/h/w match` against ALL videos regardless of document_id. The new shim computes `dedup\_key='tg:fp:S:D:H:W'` and looks up by that key. But a prior video WITH a document_id was stored with `dedup\_key='tg:doc:X'`, so the fingerprint lookup misses it.

Let me see the rest of telethon_client.py insert path (the IntegrityError handling) and read the test files briefly.

12. user (2026-07-06T06:03:17.051Z)

```

262     try:
263         inserted = await db.insert_video(conn, record)
264     except sqlite3.IntegrityError:
265         # Lost a dedup race: a concurrent delivery of the same file
266         # (same document_id or same message instance) won the insert.
267         # The three-tier check above already classified this as new, so
268         # the collision means another task is processing (or has
269         # processed) it ? log and skip.
270         log.info(
271             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
272             user.id,
273             channel_id,
274             message_id,
275             document_id,
276         )
277         return
278     log.info(

```

```

279         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
280         user.id,
281         channel_id,
282         message_id,
283         result.matched,
284         result.reasons,
285     )
286
287     if result.matched:
288         # Build link text for LINK/BOTH modes (None for FORWARD).
289         link_text = None
290         if delivery_mode != DeliveryMode.FORWARD:
291             channel_title = _channel_title(event)
292             link_text = build_link_text(
293                 message_link=getattr(message, "link", None),
294                 duration_s=meta.duration_s,
295                 height=meta.height,
296                 size_bytes=meta.size_bytes,
297                 channel_title=channel_title,
298             )
299         await deliver(
300             client,
301             message,

```

13. assistant (2026-07-06T06:03:20.648Z)

Now let me write a repro to confirm the seed claim. This is the core finding ? I'll verify the regression concretely.

14. assistant (2026-07-06T06:03:31.249Z)

```

import asyncio, aiosqlite
from datetime import datetime, UTC
import sys
sys.path.insert(0, r"C:/Users/avidu/Projects/Telegram/src")
from tg_video_filter import db
from tg_video_filter.models import Account, VideoRecord

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=1, telegram_id=1, created_at=datetime.now(UTC)))

    # Prior video WITH document_id in channel 100, msg 10.
    v1 = VideoRecord(user_id=1, channel_id=100, message_id=10, document_id=99999,
        duration_s=60, width=1920, height=1080, size_bytes=5000000,
        mime_type="video/mp4", matched=True, seen_at=datetime.now(UTC))
    await db.insert_video(conn, v1)

    # A re-upload arrives: SAME fingerprint, but different (new) document_id,
    # different channel/msg. telethon_client Tier-2 checks fingerprint.
    hit = await db.get_video_by_content_fingerprint(
        conn, 1, size_bytes=5000000, duration_s=60, height=1080, width=1920)
    print("Tier-2 fingerprint lookup vs a doc-keyed prior video ->", hit)

    # Sanity: v1-main WOULD have matched (SELECT WHERE size/dur/h/w).
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        "SELECT count(*) c FROM content_items WHERE size_bytes=? AND duration_s=? AND height=?
AND width=?",
        (5000000, 60, 1080, 1920)) as cur:
        r = await cur.fetchone()
    print("rows physically matching the fingerprint columns:", r["c"])

    # Also check: what dedup_key did the prior video get stored under?

```

```

async with conn.execute("SELECT dedup_key FROM content_items") as cur:
    print("stored dedup_keys:", [row["dedup_key"] for row in await cur.fetchall()])

# And what key does the fingerprint shim compute/look up?
print("fingerprint shim looks up key:",
      db.compute_dedup_key(provider="telegram", size_bytes=5000000, duration_s=60,
height=1080, width=1920))

asyncio.run(main())

```

15. user (2026-07-06T06:03:31.275Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py (file state is current in your context ? no need to Read it back)

16. user (2026-07-06T06:03:40.804Z)

Tier-2 fingerprint lookup vs a doc-keyed prior video -> None
rows physically matching the fingerprint columns: 1
stored dedup_keys: ['tg:doc:99999']
fingerprint shim looks up key: tg:fp:5000000:60:1080:1920

17. assistant (2026-07-06T06:03:45.885Z)

The seed claim is CONFIRMED. The prior video was stored under `tg:doc:99999`, but the fingerprint shim looks up `tg:fp:5000000:60:1080:1920` ? a miss, even though a row physically matches all four fingerprint columns. In v1 (main), `get\_video\_by\_content\_fingerprint` did `SELECT WHERE size/dur/h/w match` and would have found it.

Now let me assess the real-world severity via the caller in telethon_client.py. The Tier-2 check runs AFTER Tier-1 (document_id lookup). Let me trace the exact scenario where this causes a duplicate to slip through, and confirm insert doesn't catch it either.

18. assistant (2026-07-06T06:03:55.549Z)

```

import asyncio, aiosqlite, sqlite3
from datetime import datetime, UTC
import sys
sys.path.insert(0, r"C:/Users/avidu/Projects/Telegram/src")
from tg_video_filter import db
from tg_video_filter.models import Account, VideoRecord

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, Account(id=1, telegram_id=1, created_at=datetime.now(UTC)))

    # Prior video WITH document_id A.
    v1 = VideoRecord(user_id=1, channel_id=100, message_id=10, document_id=111,
                      duration_s=60, width=1920, height=1080, size_bytes=5000000,
                      mime_type="video/mp4", matched=True, seen_at=datetime.now(UTC))
    await db.insert_video(conn, v1)

    # Re-upload: identical bytes/metadata, DIFFERENT new document_id B, different channel.
    # Simulate telethon_client's tier chain:
    doc_b = 222
    # Tier 0
    t0 = await db.get_video(conn, 1, 200, 20)
    # Tier 1
    t1 = await db.get_video_by_document_id(conn, 1, doc_b)
    # Tier 2
    t2 = await db.get_video_by_content_fingerprint(conn, 1, 5000000, 60, 1080, 1920)
    print("Tier0:", t0, "Tier1:", t1, "Tier2:", t2)

```



```
raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

21. assistant (2026-07-06T06:04:11.464Z)

Fully confirmed (the trailing traceback is just aiosqlite's worker thread noise on interpreter exit, not part of the result). The re-upload with a new document_id passes all three tiers as "new", the insert SUCCEEDS (no constraint stops it because `tg:doc:222` ? `tg:doc:111`), and a ****duplicate video is stored and delivered****. In v1 this was caught by Tier-2.

This is the headline regression. Now let me check the other investigation points: parity between Python `compute\_dedup\_key` and the SQL CASE, provider casing, and empty-string document_id, plus the migration's own dedup-key computation. Let me check one subtle parity issue ? the migration SQL uses `s.id` (the newly-created source id) for the Tier-0 fallback, whereas the Python function in `insert\_video` uses `src.id`. That's consistent. But let me check provider casing and the `get\_video\_by\_document\_id` shim behavior for the empty-string document edge.

Let me also verify: does the migration produce the SAME key as the shim for a doc-keyed row? Migration uses `tg:doc:` || v.document_id` (document\_id is INTEGER in videos), and shim uses `compute_dedup_key(document_id=str(...))` ? `tg:doc:{document_id}`. For document\_id=99999 both give `tg:doc:99999`. Consistent.

Let me verify provider casing parity concern in the SQL vs Python, and check the test file to see what's actually tested.

22. user (2026-07-06T06:04:13.421Z)

```
252:async def test_insert_video_dedup_collision(conn: aiosqlite.Connection) -> None:
295:async def test_video_record_dedup_key() -> None:
297:    assert v.dedup_key == (1, 2, 3)
300:async def test_video_record_content_fingerprint() -> None:
310:    assert v.content_fingerprint == (5_000_000, 120, 1080, 1920)
313:async def test_video_record_content_fingerprint_none_when_missing_field() -> None:
324:    assert v.content_fingerprint is None
327:# --- content dedup (Tiers 1 & 2) -----
353:    """Same document_id for the same user ? IntegrityError (Tier-1 dedup at DB level)."""
402:async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
416:    found = await db.get_video_by_content_fingerprint(
428:async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) -> None:
432:    await db.get_video_by_content_fingerprint(
612:    superseded by ``UNIQUE(account_id, dedup_key)`` on ``content_items``; the
617:    # The provider-neutral dedup uniqueness lives on content_items, not videos.
621:    # at least one unique index exists (the dedup_key uniqueness).
622:    assert any("dedup_key" in idx or "sqlite_autoindex" in idx for idx in idx_names,
idx_names
752:        "duration_s, width, height, size_bytes, dedup_key FROM content_items"
755:        # dedup_key falls to the Tier-0 branch (no document_id, but the
756:        # fingerprint fields are all present) ? actually the fingerprint branch
764:        assert ci[8] == "tg:fp:1000000:60:1080:1920" # fingerprint dedup_key
848:async def test_document_id_unique_index_enforces_dedup(conn: aiosqlite.Connection) -> None:
874:# --- compute_dedup_key (ADR 0011, Option 1A) -----
877:def test_compute_dedup_key_telegram_document_id_wins() -> None:
879:    key = db.compute_dedup_key(
889:    assert key == "tg:doc:12345"
892:def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
893:    """Tier-2 (fingerprint) applies when document_id is absent AND all four
894:    metadata fields are present (the guard against degenerate tg:fp:0:0:0:0)."""
895:    key = db.compute_dedup_key(
902:    assert key == "tg:fp:1000:60:1080:1920"
905:def test_compute_dedup_key_telegram_tier0_when_partial_metadata() -> None:
906:    """Tier-0 (message id) applies when fingerprint is incomplete ? prevents
908:    key = db.compute_dedup_key(
915:        width=None, # missing ? fingerprint branch skipped
920:def test_compute_dedup_key_telegram_tier0_when_all_metadata_missing() -> None:
922:    key = db.compute_dedup_key(provider="telegram", source_id=7, provider_message_id="42")
```

```

926: def test_compute_dedup_key_telegram_empty_document_id_falls_through() -> None:
928:     key = db.compute_dedup_key(
937: def test_compute_dedup_key_telegram_tier0_requires_source_and_message() -> None:
941:     db.compute_dedup_key(provider="telegram") # no metadata at all
944: def test_compute_dedup_key_other_provider_uses_provider_prefix() -> None:
946:     key = db.compute_dedup_key(
952: def test_compute_dedup_key_key_other_provider_requires_source_and_message() -> None:
955:     db.compute_dedup_key(provider="whatsapp")
958: def test_compute_dedup_key_provider_is_case_insensitive() -> None:
961:     db.compute_dedup_key(provider="TELEGRAM", document_id="5")
962:     == db.compute_dedup_key(provider="telegram", document_id="5")
963:     == "tg:doc:5"
1037:     dedup_key=db.compute_dedup_key(provider="telegram", document_id="999"),
1044:     by_key = await db.get_content_item_by_dedup_key(conn, USER_ID, "tg:doc:999")
1049: async def test_content_item_dedup_key_uniqueness(conn: aiosqlite.Connection) -> None:
1050:     """UNIQUE(account_id, dedup_key) rejects a second item with the same key."""
1066:     dedup_key="tg:doc:777",
1076:         provider_message_id="2", # different message, same dedup key
1078:         dedup_key="tg:doc:777",
1119: async def test_migration_003_backfills_all_dedup_key_branches(tmp_path: object) -> None:
1120:     """The content_items backfill must exercise all three dedup_key branches
1121:     (document_id / fingerprint / Tier-0 message) and never collapse two rows
1124:     db_path = str(tmp_path) + "/legacy_dedup.db" # type: ignore[operator]

```

23. assistant (2026-07-06T06:04:17.518Z)

Let me look at the current test for `get\_video\_by\_content\_fingerprint\_hit/miss` to see whether the test masks the regression (i.e., whether the test only seeds a video WITHOUT a document_id).

24. user (2026-07-06T06:04:18.595Z)

```

402     async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
403         await _seed_user(conn)
404         await db.insert_video(
405             conn,
406             VideoRecord(
407                 user_id=USER_ID,
408                 channel_id=CHANNEL_ID,
409                 message_id=5,
410                 size_bytes=10_000_000,
411                 duration_s=180,
412                 height=720,
413                 width=1280,
414             ),
415         )
416         found = await db.get_video_by_content_fingerprint(
417             conn,
418             USER_ID,
419             10_000_000,
420             180,
421             720,
422             1280,
423         )
424         assert found is not None
425         assert found.message_id == 5
426
427
428     async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) ->
None:
429         await _seed_user(conn)
430         # No video inserted ? should be None.
431         assert (
432             await db.get_video_by_content_fingerprint(
433                 conn,
434                 USER_ID,

```

```
435         1,
436         1,
437         1,
438         1,
439     )
440     is None
441 )
```

25. assistant (2026-07-06T06:04:29.807Z)

Confirmed the test masks the regression: the "hit" test seeds a video with NO `document\_id`, so the prior row is stored under `tg:fp:...` and the fingerprint lookup finds it. The regressed case (prior video HAS a document_id) is never tested. This is exactly the seed claim.

Now let me quickly check the other investigation dimensions for additional findings. Let me look at the `insert\_video` shim's handling and confirm there's no second issue with cross-source Tier-0. Let me also verify the provider casing in the SQL migration is fine (it hardcodes 'telegram', so no casing issue there).

Let me check one more thing: `get\_video\_by\_document\_id` shim recovers `channel\_id` via `int(src.provider\_source\_id)` ? fine. And confirm the Tier-1 path is NOT regressed (doc?doc still works). My repro already showed Tier-1 hit works for same doc. Let me verify the document_id Tier-1 is intact and there are no additional dedup bugs by checking the `\_seed\_user` and CHANNEL constants.

I have enough to report. The core finding is the Tier-2 regression. Let me also assess whether there's a genuine secondary concern: the `insert\_video` shim computes dedup_key with document_id present ? `tg:doc:X`, meaning the `(account\_id, dedup\_key)` UNIQUE constraint no longer enforces fingerprint uniqueness at all (v1 only had a doc_id unique index, so v1 ALSO didn't enforce fingerprint uniqueness at DB level ? it relied on the read-path Tier-2 check). So the DB constraint parity is fine; the regression is purely in the read-path shim.

Let me confirm one nuance about real-world severity: the ADR itself claims "Existing `videos` rows migrate cleanly" and README advertises 3-tier dedup. The regression means Tier-2 is effectively dead for the overwhelmingly common case (videos with document_id). Let me verify the README claim.

26. user (2026-07-06T06:04:33.477Z)

```
README.md:31:- **Never forwards the same video twice** ? three-tier content dedup
README.md:32: (message instance ? Telegram `document.id` ? metadata fingerprint) means
docs/design.md:56:? dedup 3-tier (ch,msg,doc,fp) ? ? '@user' / chat id) ?
docs/design.md:130:**Tier-2 content dedup** uses a metadata fingerprint
docs/design.md:220:4. Run the **three-tier dedup** check (cheapest first; see ADR 0006):
docs/design.md:224: - **Tier 2**: `(size_bytes, duration_s, height, width)` fingerprint ?
docs/design.md:225: re-upload of an identical file with a new `document.id`.
```

27. assistant (2026-07-06T06:04:37.387Z)

The regression is fully confirmed and independently corroborated. The README explicitly advertises "Never forwards the same video twice ? three-tier content dedup", and design.md §130/§224 documents Tier-2 as catching "re-upload of an identical file with a new document.id" ? which is exactly the case this PR breaks.

I have everything needed. Let me compile findings.

28. user (2026-07-06T06:04:56.396Z)

Structured output provided successfully